Algoritmo de Deutsch

Genesis Silesky

21 de octubre de 2026

1. Algoritmo de Deutsch

1.1. Objetivo

Resolver con una sola consulta un problema que clásicamente necesita dos: decidir si una función de un bit es constante o balanceada.

1.2. Conceptos previos

Concepto previo: superposición, puerta H e interferencia

Ver capítulos 01, 02 y 05a. La puerta H convierte $|0\rangle$ en $(|0\rangle + |1\rangle)/\sqrt{2}$ y $|1\rangle$ en $(|0\rangle - |1\rangle)/\sqrt{2}$, y dos H seguidas se cancelan.

1.3. Fundamento teórico

El algoritmo de Deutsch, propuesto por David Deutsch en 1985, se considera el primer algoritmo que demuestra la ventaja de un computador cuántico sobre uno clásico.

El problema (las cajas de pintura). Hay dos bolas, una liviana ($x = 0$) y una pesada ($x = 1$), que se pintan de blanco ($f = 0$) o negro ($f = 1$). Hay cajas de un solo color (pintan las dos bolas igual) y cajas de dos colores (las pintan distinto), pero se mezclaron. ¿Cuántas veces hay que pintar para distinguirlas?

En términos matemáticos, la caja es una función $f : \{0, 1\} \to \{0, 1\}$:

- **constante** si $f(0) = f(1)$ (caja de un solo color);
- **balanceada** si $f(0) \neq f(1)$ (caja de dos colores).

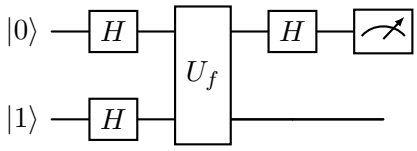
Clásicamente hay que evaluar $f(0)$ y $f(1)$ y compararlos: **dos consultas**.

El oráculo. La función se da como una puerta unitaria U_f que actúa como

$$U_f \,|x\rangle \,|y\rangle = |x\rangle \,|y \oplus f(x)\rangle \,, \tag{1}$$

donde $\oplus$ es la suma módulo 2 (XOR). El primer qubit es el *registro de datos* y el segundo el *registro objetivo*. Gracias a la superposición, una sola aplicación de U_f evalúa $f(0)$ y $f(1)$ a la vez (*paralelismo cuántico*).

El circuito.



1. Estado inicial $|\psi_0\rangle = |0\rangle\,|1\rangle$.
2. Con H en ambos qubits: $|\psi_1\rangle = \frac{|0\rangle+|1\rangle}{\sqrt{2}}\,\frac{|0\rangle-|1\rangle}{\sqrt{2}}$.
3. Una consulta a U_f. Como el objetivo está en $(|0\rangle - |1\rangle)/\sqrt{2}$, el oráculo solo cambia el signo de cada $|x\rangle$:
$$U_f\,|x\rangle\,\frac{|0\rangle - |1\rangle}{\sqrt{2}} = (-1)^{f(x)}\,|x\rangle\,\frac{|0\rangle - |1\rangle}{\sqrt{2}}. \tag{2}$$
Por tanto $|\psi_2\rangle = \frac{(-1)^{f(0)}|0\rangle+(-1)^{f(1)}|1\rangle}{\sqrt{2}}\,\frac{|0\rangle-|1\rangle}{\sqrt{2}}$.
   - Si f es constante, los datos quedan en $\pm\frac{|0\rangle+|1\rangle}{\sqrt{2}}$.
   - Si f es balanceada, quedan en $\pm\frac{|0\rangle-|1\rangle}{\sqrt{2}}$.
4. H sobre los datos: $|\psi_3\rangle = \pm\,|0\rangle$ si f es constante y $\pm\,|1\rangle$ si es balanceada.
5. Se mide el registro de datos: 0 significa constante y 1 significa balanceada.

Una consulta contra dos.

1.4. Código (completamente comentado)

```python
# %% [markdown]
# # 15 - Algoritmo de Deutsch: una consulta en vez de dos

# %% Celda 1: importaciones
from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator
from qiskit.quantum_info import Statevector

sim = AerSimulator()

# %% Celda 2: el problema (las cajas de pintura)
# Una funcion f(x) con x en {0, 1} es una "caja negra". Hay cuatro posibles:
#   constante: f(0) = f(1)  (siempre 0, o siempre 1)    -> caja de un solo color
#   balanceada: f(0) != f(1)  (identidad o negacion)    -> caja de dos colores
# Clasicamente hay que preguntar dos veces (f(0) y f(1)) para saberlo.
funciones = {
    "constante 0": (0, 0),
    "constante 1": (1, 1),
    "balanceada f(x)=x": (0, 1),
    "balanceada f(x)=1-x": (1, 0),
}
for nombre, (f0, f1) in funciones.items():
    tipo = "constante" if f0 == f1 else "balanceada"
    print(f"{nombre:22s}: f(0) = {f0}, f(1) = {f1} -> {tipo}")


# %% Celda 3: el oraculo Uf
# El oraculo es una puerta unitaria que actua asi:  Uf |x>|y> = |x>|y xor f(x)>.
# q0 es el registro de datos (x) y q1 el registro objetivo (y).
```

```python
# Se construye aplicando una X al objetivo cuando f(x) = 1:
def oraculo(f0, f1):
    qc = QuantumCircuit(2)
    if (f0, f1) == (1, 1):
        qc.x(1)                         # f(x) = 1 siempre: se invierte siempre
    elif (f0, f1) == (0, 1):
        qc.cx(0, 1)                     # f(x) = x: se invierte si x = 1
    elif (f0, f1) == (1, 0):
        qc.cx(0, 1)                     # f(x) = 1 - x: se invierte si x = 0
        qc.x(1)
    return qc                           # (0, 0): no se hace nada


# %% Celda 4: el circuito de Deutsch
#   1. Datos en |0> y objetivo en |1>.
#   2. H en los dos: datos en (|0>+|1>)/sqrt2 y objetivo en (|0>-|1>)/sqrt2.
#   3. Una sola consulta al oraculo Uf.
#   4. H en el registro de datos y se mide.
# Si sale 0, f es constante; si sale 1, f es balanceada.
def deutsch(f0, f1):
    qc = QuantumCircuit(2, 1)
    qc.x(1)                    # objetivo en |1>
    qc.h([0, 1])
    qc.compose(oraculo(f0, f1), inplace=True)
    qc.h(0)
    qc.measure(0, 0)
    return qc


# En Colab o Jupyter el circuito se dibuja solo; en terminal no se muestra.
deutsch(0, 1).draw()

# %% Celda 5: ejecutar con las cuatro funciones
# Una sola consulta basta, y el resultado es siempre el mismo (no es aleatorio).
for nombre, (f0, f1) in funciones.items():
    cuentas = sim.run(deutsch(f0, f1), shots=1000).result().get_counts()
    dato = max(cuentas, key=cuentas.get)
    veredicto = "constante" if dato == "0" else "balanceada"
    print(f"{nombre:22s}: medicion = {cuentas} -> {veredicto}")

# %% Celda 6: por que funciona (cambio de signo por la fase)
# Con el objetivo en (|0>-|1>)/sqrt2, el oraculo solo cambia el signo de cada x:
#   Uf |x> (|0>-|1>)/sqrt2 = (-1)^f(x) |x> (|0>-|1>)/sqrt2.
# El registro de datos queda en ((-1)^f(0)|0> + (-1)^f(1)|1>)/sqrt2:
#   constante  -> (|0>+|1>)/sqrt2  (o su opuesto)  -> la H final lo lleva a |0>
#   balanceada -> (|0>-|1>)/sqrt2  (o su opuesto)  -> la H final lo lleva a |1>
for nombre, (f0, f1) in funciones.items():
    qc = QuantumCircuit(2)
    qc.x(1)
    qc.h([0, 1])
    qc.compose(oraculo(f0, f1), inplace=True)
    amplitudes = Statevector(qc).data
    # El objetivo (q1) vale (|0>-|1>)/sqrt2, asi que las amplitudes con q1 = 0 (indices
    # 0 y 1) son las de los datos multiplicadas por 1/sqrt2. Multiplicando por 2 queda
    # +-1, que son los signos (-1)^f(x) de la formula.
    a0, a1 = amplitudes[0] * 2, amplitudes[1] * 2
    print(f"{nombre:22s}: datos = ({a0.real:+.0f}|0> {a1.real:+.0f}|1>)/sqrt2")
```

1.5. Explicación celda por celda

Celda 2 Lista las cuatro funciones posibles y dice si cada una es constante o balanceada.

Celda 3 Construye el oráculo U_f: una X al objetivo si f vale 1 siempre, una CNOT si $f(x) = x$, y CNOT más X si $f(x) = 1 - x$.

Celda 4 Arma el circuito: objetivo en $|1\rangle$, H en ambos, oráculo, H en los datos y medición.

Celda 5 Lo ejecuta con las cuatro funciones y 1000 mediciones.

Celda 6 Muestra los signos $(-1)^{f(x)}$ del registro de datos después del oráculo.

1.6. Salida obtenida

```
constante 0           : f(0) = 0, f(1) = 0 -> constante
constante 1           : f(0) = 1, f(1) = 1 -> constante
balanceada f(x)=x     : f(0) = 0, f(1) = 1 -> balanceada
balanceada f(x)=1-x   : f(0) = 1, f(1) = 0 -> balanceada
constante 0           : medicion = {'0': 1000} -> constante
constante 1           : medicion = {'0': 1000} -> constante
balanceada f(x)=x     : medicion = {'1': 1000} -> balanceada
balanceada f(x)=1-x   : medicion = {'1': 1000} -> balanceada
constante 0           : datos = (+1|0> +1|1>)/sqrt2
constante 1           : datos = (-1|0> -1|1>)/sqrt2
balanceada f(x)=x     : datos = (+1|0> -1|1>)/sqrt2
balanceada f(x)=1-x   : datos = (-1|0> +1|1>)/sqrt2
```

(El dibujo del circuito no se reproduce aquí.)

1.7. Interpretación

El resultado no es aleatorio: las dos funciones constantes siempre dan 0 y las dos balanceadas siempre dan 1, con una sola consulta. Los signos de la última parte coinciden con $(-1)^{f(x)}$: $(+1, +1)$ y $(-1, -1)$ para las constantes, $(+1, -1)$ y $(-1, +1)$ para las balanceadas. Lo que distingue los casos es la *fase relativa* entre $|0\rangle$ y $|1\rangle$, que la H final convierte en una diferencia medible (interferencia, capítulo 05a).

1.8. Ejercicios

1. Elimina la X inicial del objetivo y mira qué cambia en la Celda 6.
2. ¿Cuántas consultas necesitaría un algoritmo clásico para $n = 1$? ¿Y el cuántico?
3. Cambia el oráculo de $f(x) = x$ por otro circuito equivalente y comprueba que el resultado no cambia.

1.9. Errores comunes

- Creer que el algoritmo conoce $f(0)$ y $f(1)$ por separado: solo averigua si son iguales o distintos.
- Olvidar poner el objetivo en $|1\rangle$ antes de la H: sin esa fase el oráculo no cambia el signo.